

Validation & Exception Handling

Dr. Öğr. Üyesi Alper Talha KARADENİZ

@Valid

- @Valid anotasyonu, bir nesnenin otomatik olarak doğrulanması gerektiğini belirtir. Genellikle bir denetleyici (controller) metodunun parametresine eklenir. Spring MVC, bu anotasyonu gördüğünde, gelen HTTP istek verilerini nesneye bağladıktan sonra ancak metodu çağırmadan önce doğrulama kurallarını çalıştırır. Eğer nesne içinde tanımlanan kurallara uymayan bir veri varsa, hata detayları metoda iletilen bir Errors nesnesinde toplanır ve geliştirici bu hatalara göre işlem yapabilir.

@NotNull

- @NotNull, bir alanın (field) değerinin boş (null) olmamasını garanti altına almak için kullanılan temel bir kuraldır. Örneğin, bir Taco nesnesi oluşturulurken isminin mutlaka girilmiş olması gerekiyorsa, ilgili alan bu anotasyonla işaretlenir. Genellikle @Size gibi kısıtlayıcı anotasyonlarla birlikte kullanılarak verinin hem var olması hem de belirli bir uzunlukta olması sağlanır.

Global Exception Handling

- Global Exception Handling (Küresel İstisna Yönetimi), bir uygulamadaki istisnaların (exceptions) her denetleyici (controller) metodunda ayrı ayrı yönetilmesi yerine, merkezi bir noktadan yakalanıp işlenmesini sağlayan bir yaklaşımdır. Spring Framework, bu süreci kolaylaştırmak için özellikle REST servislerinde kullanılan güçlü mekanizmalar sunar.

@RestControllerAdvice Kullanımı

- Küresel istisna yönetiminin merkezinde @RestControllerAdvice anotasyonu yer alır. Bu anotasyon ile işaretlenen bir sınıf, bir "aspect" (bakış açısı) görevi görerek tüm denetleyicilerden fırlatılan istisnaları takip eder.
- Avantajı: İstisna mantığını iş mantığından ayırarak kodun daha temiz, bakımı kolay ve okunabilir olmasını sağlar. Ayrıca, aynı hata türünü birden fazla uç nokta (endpoint) için tek bir yerden yöneterek kod tekrarını (boilerplate) önler.

@ExceptionHandler Metotları

- @RestControllerAdvice sınıfı içinde, belirli istisna türlerini yakalamak için @ExceptionHandler anotasyonu ile işaretlenmiş metotlar (exception handlers) tanımlanır.
- Özellikleri: Bu metotlar, parametre olarak yakalanan istisna nesnesini alabilir, böylece hatanın detaylarına erişebilirler. Genellikle istemciye anlamlı bir hata mesajı ve uygun bir HTTP durumu dönerler.

ResponseEntity ile Yanıt Yönetimi

- İstisna yönetimi sırasında istemciye gönderilecek yanıtın gövdesi, başlıkları ve HTTP durum kodu (örneğin; "400 Bad Request", "404 Not Found") ResponseEntity sınıfı kullanılarak özelleştirilir.
- Hata Detayları (DTO): Genellikle sadece bir hata mesajı içeren basit bir sınıf (örneğin ErrorDetails) oluşturulur ve bu nesne JSON formatında yanıt gövdesine eklenir.

@ControllerAdvice

- @ControllerAdvice (ve kaynaklarda REST servisleri bağlamında sunulan özel haliyle @RestControllerAdvice), Spring uygulamalarında denetleyicilerden (controllers) fırlatılan istisnaları (exceptions) merkezi bir noktadan yakalamak ve yönetmek için kullanılan bir aspect (bakış açısı) sınıfıdır.

Temel İşlevleri ve Avantajları

- **Sorumlulukların Ayrılması:** İstisna yönetimi mantığını, iş mantığının (business logic) uygulandığı denetleyici sınıflarından ayırır. Bu sayede denetleyici metotları sadece "mutlu yolu" (happy flow) işleyebilirken, hata durumları bu merkezi sınıfta ele alınır.
- **Merkezi Hata Yönetimi:** Birden fazla uç noktada (endpoint) ortaya çıkabilecek aynı türden hataların her metotta ayrı ayrı yakalanması (try-catch) yerine, tek bir sınıfta yönetilmesini sağlayarak kod tekrarını (boilerplate code) önler.

Temel İşlevleri ve Avantajları

- **@ExceptionHandler Metotları:** @ControllerAdvice ile işaretlenmiş sınıfın içinde, belirli istisna türlerini yakalamak için @ExceptionHandler anotasyonu ile işaretlenmiş metotlar tanımlanır. Bu metotlar, örneğin bir "Bakiye Yetersiz" hatası fırlatıldığında devreye girerek istemciye anlamlı bir hata mesajı ve uygun bir HTTP durum kodu döner.
- **REST Servisleri İçin Özelleşme:** Kaynaklarda vurgulanan @RestControllerAdvice, denetleyicilerden dönen verilerin doğrudan HTTP yanıt gövdesine (response body) yazılmasını sağlar. Bu, @ControllerAdvice ve @ResponseBody anotasyonlarının birleşimi olarak işlev görür.

Spring Framework'te Durum Kodlarını Yönetme

- **ResponseEntity:** Bu sınıf kullanılarak yanıt gövdesi, başlıkları ve durum kodu manuel olarak özelleştirilebilir.
- **@ResponseStatus:** Bir metodun başarılı bir şekilde tamamlanması durumunda döneceği özel kod (örneğin 201 veya 204) bu anotasyonla sabitlenebilir.
- **onStatus():** Reaktif programlamada (WebClient), belirli hata kodları (4xx veya 5xx) algılandığında özel hata yönetimi mantığı çalıştırmak için kullanılır.